Lista de Exercícios – Funções em Clojure (com Gabarito)

1■■ Função de Boas-Vindas

Crie uma função `saudar` que receba um nome e retorne uma mensagem de saudação personalizada.

```
(ns ex1.core)

(defn saudar [nome]
  (str "Olá, " nome "! Seja bem-vindo(a)."))

(defn -main []
  (println (saudar "Lucas")))
```

Saída esperada:
Olá, Lucas! Seja bem-vindo(a).

2■■ Quadrado de um Número

Crie uma função `quadrado` que receba um número e retorne seu valor ao quadrado. Implemente também uma versão anônima utilizando `#()`.

```
(ns ex2.core)

(defn quadrado [n]
  (* n n))

(def quadrado-anonimo #(* % %))

(defn -main []
  (println "Quadrado (defn):" (quadrado 4))
  (println "Quadrado (anonimo):" (quadrado-anonimo 4)))
```

Saída esperada:
Quadrado (defn): 16
Quadrado (anonimo): 16

3■■ Soma de uma Lista (Recursão)

Crie uma função `soma-lista` que receba uma lista e calcule a soma dos seus elementos usando recursão.

```
(ns ex3.core)

(defn soma-lista [lst]
  (if (empty? lst)
    0
    (+ (first lst) (soma-lista (rest lst)))))

(defn -main []
  (println "Soma da lista:" (soma-lista '(1 2 3 4 5))))
```

Saída esperada:
Soma da lista: 15

4■■ Dobrar e Somar (com encadeamento ->>)

Dada uma lista de números, dobre cada elemento e calcule a soma total usando encadeamento `->>`.

```
(ns ex4.core)

(defn -main []
  (let [numeros [1 2 3 4 5]
        resultado (->> numeros
                       (map #(* 2 %))
                       (reduce +))]
    (println "Soma dos valores dobrados:" resultado)))
```

Saída esperada:
Soma dos valores dobrados: 30

5■■ Função Composta (comp)

Crie duas funções: `dobrar` e `incrementar`. Use `comp` para criar uma nova função `dobrar-e-incrementar`.

```
(ns ex5.core)

(defn dobrar [n] (* n 2))
(defn incrementar [n] (inc n))
(def dobrar-e-incrementar (comp incrementar dobrar))

(defn -main []
  (println (dobrar-e-incrementar 5)))
```

Saída esperada:
11

6■■ Processar Notas (Funções + Encadeamento + Filter)

Crie um programa que receba uma lista de notas, some 1 ponto em cada nota, filtre apenas as notas maiores ou iguais a 6, calcule a média final e exiba o resultado.

```
(ns ex6.core)

(defn adicionar-bonus [nota] (+ nota 1))
(defn aprovado? [nota] (>= nota 6))
(defn media [notas] (double (/ (reduce + notas) (count notas))))

(defn -main []
  (let [notas [5.0 7.5 4.0 9.0 6.5]
        resultado (->> notas
                       (map adicionar-bonus)
                       (filter aprovado?)
                       (doall))
```

```
        media-final (media resultado)]
    (println "Notas originais:" notas)
    (println "Notas com bônus:" (map adicionar-bonus notas))
    (println "Aprovados:" resultado)
    (println "Média dos aprovados:" media-final)))
```

Saída esperada:
Notas originais: [5.0 7.5 4.0 9.0 6.5]
Notas com bônus: (6.0 8.5 5.0 10.0 7.5)
Aprovados: (6.0 8.5 10.0 7.5)
Média dos aprovados: 8.0